

Natural-Language Reasoning AI Tools for Mathematics

Usage pattern before implementation details

DSA5210 – Session 4

Use Means Delegating a Workflow

Bad usage pattern:

Prove this theorem.

- The statement may be incomplete.
- The tool may need local setup.
- The output may have a limited verification status.

Better usage pattern:

Formulate, inspect, run, and interpret.

- Stabilize the mathematical target.
- Recover the local tool contract.
- Report artifacts and claim boundary.

The Usage Pattern



Problem

What exactly is the mathematical task?

Operation

How does this local system expect to be run?

Output

What does the artifact actually justify?

Problem Formulation Comes First

Paper-assistant style

- ambient setting;
- definitions and conventions;
- known inputs;
- target theorem;
- stronger and weaker variants.

Problem-constructor style

- background and notation;
- allowed inputs;
- target claims;
- required proof details;
- completeness check.

The first useful output is a solver-ready problem file, not a proof attempt.

Prompt: Formulate the Mathematical Task

Use my problem formulation workflow before using any proof tool.

First, use a paper-assistant style pass:

- stabilize the ambient setting and notation;
- separate definitions, known inputs, conjectures, and target claims;
- identify missing hypotheses or ambiguous conventions.

Then use a problem-constructor style pass:

- produce a self-contained solver-ready Markdown problem file;
- include background, allowed inputs, target claims, proof details, a completeness check request, and expected output;
- do not try to prove the theorem yet.

Raw material:

<paste notes, theorem sketch, conjecture, or project excerpt here>

Case Study

Codex runs Rethlas

Rethlas Is the Local Tool

Demo checkout:

`~/mydoc/hallproblem/Rethlas`

Problem file:

`agents/generation/data/hall_theorem.md`

Rethlas is a natural-language proof-search framework with a generator, verifier, memory, logs, and proof blueprints.

The lecture uses Rethlas as an example of the usage pattern, not as the whole topic of L4.

Prompt: Run Rethlas

Use Codex to run Rethlas for this problem.

Checkout:

~/mydoc/hallproblem/Rethlas

Problem file:

agents/generation/data/hall_theorem.md

First inspect the local implementation. Recover the launch contract from README.md, AGENTS.md, run_example.sh, .codex/config.toml, the MCP server, and the verification API.

Then start the required processes, monitor the run, and report the paths of the verified blueprint, verification JSON, logs, and memory.

Do not read or print secrets.

Codex Inspects the Local Tool

Generation side

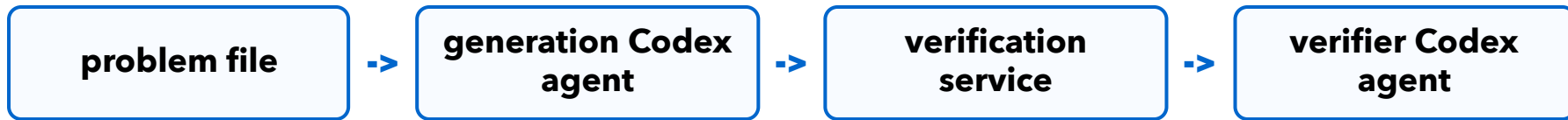
- `agents/generation/AGENTS.md`
- `agents/generation/tests/run_example.sh`
- `agents/generation/.codex/config.toml`
- `agents/generation/mcp/server.py`

Verification side

- `agents/verification/AGENTS.md`
- `agents/verification/api/server.py`
- `agents/verification/.codex/config.toml`
- `agents/verification/results/`

Local inspection first, operation second.

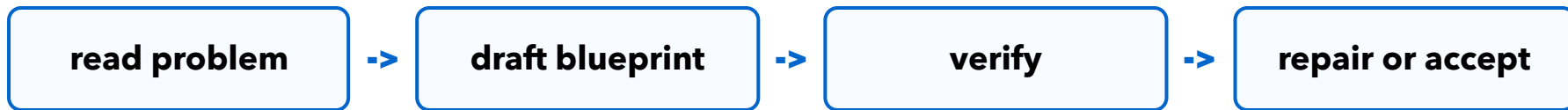
Generator and Verifier Are Different Roles



The generator drafts a proof blueprint, writes logs and memory, and calls the verifier on a complete draft.

The verifier checks the proof in order and returns structured feedback.

The Proof Loop



Rethlas is useful as a natural-language reasoning tool because it records a loop: draft, verify, repair, and preserve state.

The verifier is part of the search loop, not only an end-of-run label.

What Codex Reports Back

`blueprint_verified.md`

Proof blueprint accepted by the verifier.

`logs/hall_theorem/`

Human-readable run trajectory.

`verification.json`

Structured verifier judgment.

`memory/hall_theorem/`

Durable proof-search state.

Boundary of the Claim

Rethlas output:

natural-language proof blueprint accepted by a verifier agent.

- structured proof text;
- verification JSON;
- logs and memory.

Not the same as:

a Lean kernel certificate.

- No formal proof term is checked by Lean here.
- The verification status is agentic and natural-language.

Recording the Usage Demo

Input

`data/hall_theorem.md`

Operation

Codex inspects and runs
Rethlas.

Output

Blueprint, JSON, logs,
memory.

Use a completed run if a fresh run is slow, and say explicitly that the artifacts are from a previous Rethlas run.

Next

How does Codex become a generator in one folder and a verifier in another?